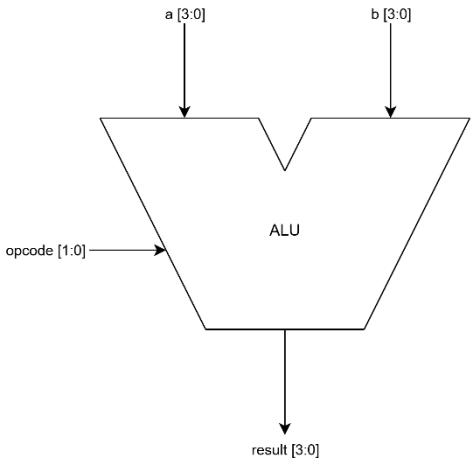
Verification Plan For 4-Bit ALU



- 4-bit ALU
- One 4-bit operand loaded from “a” port
- One 4-bit operand loaded from “b” port
- If the opcode is 2’b00 the result is the sum of the two operands
- If the opcode is 2’b01 the result is the subtraction of the two operands
- If the opcode is 2’b10 the result is the bitwise and of the two operands
- If the opcode is 2’b11 the result is the bitwise xor of the two operands
- The alu wraps around if an overflow happens

Verification Plan

Signals:

1) **A[3:0]** -> The first operand input signal
2) **B[3:0]** -> The second operand input signal
3) **Opcode[1:0]** -> Controls what operation is done
   Set to 2’b00 -> the alu calculates the sum to the result[3:0] signal
   Set to 2’b01 -> the alu calculates the subtraction to the result[3:0] signal
   Set to 2’b10 -> the alu calculates the bitwise and to the result[3:0] signal
   Set to 2’b11 -> the alu calculates the bitwise xor to the result[3:0] signal
4) **Result[3:0]** -> checker: checks if the out put is the operation done on the two operands

Functional Coverage:

We need to cover all this signals:

Result -> cover all values

-> cover toggling

Opcode -> cover all operations

A -> cover all values

B -> cover all values

A \ B -> cover operations on max values

Sequences & Constrained randomization:

1) Addition sequence -> with opcode == 2'b00
2) Subtraction sequence -> with opcode == 2'b01
3) bitwise and sequence -> with opcode == 2'b10
4) bitwise xor sequence -> with opcode == 2'b11

Test scenarios:

Scenario 1: addition sequence -> adding random numbers from range [0:15]

--Goal: testing addition of numbers in range [0 to 15]

Scenario 2: subtraction sequence -> subtracting random numbers from range [0:15]

--Goal: testing subtraction of numbers in range [0 to 15]

Scenario 3: addition sequence -> adding max numbers 15 and 15

--Goal: testing addition of max values to test the overflow

Scenario 4: bitwise and sequence -> bitwise anding on random numbers from range [0:15]

--Goal: testing bitwise and operation of numbers in range [0 to 15]

Scenario 5: bitwise xor sequence -> bitwise xoring on random numbers from range [0:15]

--Goal: testing bitwise xor operation of numbers in range [0 to 15]